DOKUZ EYLÜL UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

CME 2204 ALGORITHM ANALYSIS

Assignment-1

Comparison of Tim Sort and Quick Sort

Project Report

Prepared by:

2024510014 Emre Can Aras

Lecturer:
Assistant Professor Feriştah Dalkılıç

İzmir/Buca
Date: 20/03/2026

Outline of the Report

1. INTRODUCTION ---------- **2**

2. CODE DESCRIPTION ---------- **2**

Figure 1 – Quick Sort Optimization ---------- *3*

Figure 2 – Merge Strategy ---------- *3*

3. HARDWARE SPECIFICATIONS ---------- **4**

4. PERFORMANCE COMPARISON ---------- **4**

Table 1 - Performance Comparison Table ---------- *4*

5. REFERENCES ---------- **5**

1. Introduction

In this assignment, two state-of-the-art sorting algorithms -Tim Sort and Quick Sort- are implemented and evaluated. Both algorithms are developed from scratch and their performances are measured and compared across four distinct input configurations consisting of 1,000,000 integers each: randomly ordered, semi-ordered, increasingly ordered, and decreasingly ordered sequences.

Tim Sort is a hybrid, stable sorting algorithm derived from merge sort and insertion sort, designed to efficiently handle real-world data by exploiting the existence of monotonic runs — naturally ordered subsequences — within the input [1]. The algorithm is adaptive, utilizing binary insertion sort for short runs and a stack-based merging strategy to ensure balanced merges, while achieving O(n log n) worst-case and O(n) best-case time complexity [1].

Quick Sort is a divide-and-conquer algorithm that operates by selecting a pivot element and partitioning the remaining elements into two sub-arrays — those less than the pivot and those greater — which are then sorted recursively [2]. The method compares favorably with other known sorting approaches in terms of speed, storage efficiency, and ease of implementation [2].

2. Code Description

The implementation consists of four main classes and one utility class. SortAlgorithm serves as the abstract base class, declaring an abstract sort method and providing a shared swap utility method inherited by both sorting implementations. QuickSort and TimSort both extend SortAlgorithm, overriding the sort method with their respective algorithms. Sorter acts as a wrapper class that holds a reference to a SortAlgorithm instance and delegates sorting operations to it, allowing different algorithms to be used interchangeably. FileOperations handles I/O operations, exposing two methods: one for reading an integer array from a file and one for writing a sorted array back to a file.

In the QuickSort implementation, five pivot selection strategies are supported — FIRST, LAST, MIDDLE, RANDOM, and MEDIAN — managed through a PivotStrat enum. Since FIRST and LAST strategies degenerate to $O(n^2)$ behavior on ordered inputs, a tail call optimization is applied: the smaller partition is always processed recursively while the larger partition is handled iteratively within a while loop as can be seen in Figure 1, ensuring the call stack depth remains at O(log n) and preventing stack overflow.

In the TimSort implementation, a run size of 64 is selected, as it ensures the number of runs is close to a power of two, which yields optimal merge behavior. Each run is sorted using insertion sort, after which runs are merged iteratively using a bottom-up merge strategy as can be seen in Figure 2. To avoid boundary checks during merging, Integer.MAX_VALUE is used as a sentinel value at the end of each temporary sub-array, eliminating the need for explicit index bounds validation in the inner merge loop.

```
int pivot = partition(array, left, right);

// To prevent stack overflow, we always recurse on the smaller partition
// and handle the larger partition iteratively with the while loop.
if (pivot - left < right - pivot) {
    quickSort(array, left, right: pivot - 1, pivotSelection);
    left = pivot + 1; //larger part continues the loop
} else {
    quickSort(array, left: pivot + 1, right, pivotSelection);
    right = pivot - 1; //larger part continues the loop
}
```

Figure 1 – Quick Sort Optimization

```
//Merge all the runs one by one using merge sort.
//After every iteration, double the size of the merged sub-array.
for (int size = RUN_SIZE; size < n; size *= 2) {
    for (int left = 0; left < n; left += 2 * size) {
        int mid   = Math.min(left + size - 1, n - 1);
        int right = Math.min(left + 2 * size - 1, n - 1);

        if (mid < right) {
            merge(array, left, mid, right);
        }
    }
}
```

Figure 2 – Merge Strategy

3. Hardware Specifications

All experiments were conducted on an Apple MacBook Air equipped with an Apple M1 chip, 8 GB of unified memory, and a 256 GB SSD. The operating system used was macOS 26.3.1, and the implementation was compiled and executed using JDK 25.

4. Performance Comparison

Table 1 presents the execution times of Tim Sort and Quick Sort across four input configurations, each containing 1,000,000 integers. Elapsed time is measured exclusively for the sorting operation, excluding array initialization and file I/O. All measurements are reported in milliseconds and were obtained under identical hardware and software conditions as described in Section 3.

		RANDOM INTEGERS (Read from input files)	**SEMI-ORDERED INTEGERS (Read from input files)**	**INCREASING INTEGERS (Create in code)**	**DECREASING INTEGERS (Create in code)**
Tim Sort		93 ms	48 ms	17 ms	39 ms
Quick Sort	First element	91023 ms	91363 ms	90644 ms	91043 ms
	Last element	164009 ms	207259 ms	257865 ms	258489 ms
	Middle element	17 ms	15 ms	11 ms	11 ms
	Random Element	69 ms	37 ms	36 ms	36 ms
	Median Element	2708 ms	5861 ms	12 ms	12 ms

Table 1 - Performance Comparison Table

When the results in Table 1 are examined, it is immediately observed that Tim Sort achieves its fastest execution on increasing order data with only 17 ms, which is consistent with its design philosophy of exploiting naturally ordered subsequences. Among the Quick Sort pivot strategies, the last element strategy is clearly the worst performer, running two to three times slower than the first element strategy despite both exhibiting $O(n^2)$ behavior — suggesting that last element selection produces more unbalanced partitions in practice. On the other hand, the middle element strategy stands out as the most consistent approach, executing between 11 ms and 17 ms across all input types, demonstrating that it is largely insensitive to the structure of the input data. Regarding the random pivot strategy, it shows competitive execution times compared to Tim Sort, with each performing better in different input configurations. Finally, the median element strategy exhibits the most inconsistent behavior among all strategies, dropping to 12 ms on ordered inputs while reaching up to 5,861 ms on semi-ordered data, making it the least predictable strategy overall.

Quick Sort is not an efficient way to sort 1,000,000 integers if the pivot is selected as the last or first element of the array, as can be seen from the given execution times. However, when the middle element of the array is selected as the pivot, Quick Sort executes faster than Tim Sort in every condition. Even though Tim Sort is designed to exploit naturally ordered subsequences, Quick Sort with middle element pivot still outperforms Tim Sort even on semi-ordered data, suggesting that Quick Sort with an optimal pivot selection is more efficient on real-world data. Quick Sort with a randomly selected pivot also appears to be a potentially efficient approach, as its execution times show similarity to those of Tim Sort. The median element strategy also has potential, but it does not show as much similarity to Tim Sort on randomly ordered data.

The median-of-three strategy performs optimally on ordered inputs, as the three sampled elements reliably reflect the global order. However, on random and semi-ordered inputs, the sampled elements may not be representative, occasionally resulting in unbalanced partitions and degraded performance.

For increasing and decreasing inputs, execution times of Quick sort are nearly identical across all pivot strategies, as both configurations cause maximally unbalanced partitions regardless of the pivot selection method, resulting in the same number of comparisons and swaps.

5. References

[1] T. Peters, "Tim Sort," Python mailing list, 2002. [Online]. Available: https://bugs.python.org/file4451/timsort.txt (Accessed at: 20/03/2026 19.50)

[2] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962. doi: 10.1093/comjnl/5.1.10